

RAPPORT DE TP

Sérialisation et Marshalling

Contrat JSON, versioning, Protobuf et désérialisation sécurisée (Python)

Module : Applications Réparties et Cybersécurité — Séance 7
1ère année Cybersécurité — 2025/2026

1. Introduction

Cette séance portait sur la sérialisation et le marshalling : transformer une structure de données Python en une séquence d'octets transportable (sérialisation) puis la reconstruire (désérialisation), avec une attention particulière aux risques de sécurité liés à la désérialisation de données non fiables. Le TP guidé comportait quatre parties appliquées au projet fil rouge SGDD : la définition d'un contrat JSON strict avec validation (TP7.1), le versioning JSON v1 → v2 (TP7.2), l'utilisation de Protocol Buffers et la comparaison avec JSON (TP7.3), et la formalisation d'une politique de sérialisation sûre concernant pickle (TP7.4).

2. TP7.1 — Contrat JSON et validation stricte

2.1 Spécification du contrat

Champ (Document)	Type	Obligatoire	Règles de validation
id	int	Oui	Entier positif, > 0
title	str	Oui	1-200 caractères, non vide après strip
author	str	Oui	1-100 caractères
tags	list[str]	Non	0-20 éléments, chaque tag 1-50 caractères
classification	str	Non	Allowlist : public, internal, confidential, secret. Défaut : internal
created_at	str	Non	Format ISO 8601 (YYYY-MM-DDTHH:MM:SSZ)
Champ (UserPublic)	Type	Obligatoire	Règles de validation
username	str	Oui	3-30 caractères, alphanumérique + underscore
display_name	str	Oui	1-100 caractères
role	str	Oui	Allowlist : viewer, editor, admin

2.2 Implémentation Python

Les entités Document et UserPublic sont représentées par des dataclasses. Document inclut un champ interne `_internal_score` qui n'est jamais sérialisé.

```
from dataclasses import dataclass, field, asdict
from typing import List
import json, logging

logger = logging.getLogger(__name__)

@dataclass
class Document:
    id: int
    title: str
    author: str
    tags: List[str] = field(default_factory=list)
    classification: str = "internal"
    _internal_score: float = field(default=0.0, repr=False)

@dataclass
class UserPublic:
```

```

username: str
display_name: str
role: str

EXCLUDED_FIELDS = {"_internal_score"}
ALLOWED_CLASSIFICATIONS = {"public", "internal", "confidential", "secret"}
ALLOWED_ROLES = {"viewer", "editor", "admin"}
MAX_TITLE_LEN, MAX_AUTHOR_LEN, MAX_TAGS, MAX_TAG_LEN = 200, 100, 20, 50

def serialize_document(doc: Document) -> str:
    """Sérialise un Document en JSON, sans champs sensibles."""
    data = {k: v for k, v in asdict(doc).items() if k not in EXCLUDED_FIELDS}
    return json.dumps(data, ensure_ascii=False)

def deserialize_document(raw: str) -> Document:
    """Désérialise et valide un JSON en Document (fail closed)."""
    try:
        data = json.loads(raw)
    except json.JSONDecodeError as e:
        logger.warning("JSON invalide : %s", e)
        raise ValueError("Payload invalide")

    if not isinstance(data, dict):
        raise ValueError("Payload invalide")

    errors = []
    for f in ("id", "title", "author"):
        if f not in data:
            errors.append(f"Champ obligatoire manquant : {f}")

    if "id" in data and not (isinstance(data["id"], int) and data["id"] > 0):
        errors.append("'id' doit être un entier positif")

    if "title" in data:
        if not isinstance(data["title"], str):
            errors.append("'title' doit être une chaîne")
        elif not (1 <= len(data["title"].strip()) <= MAX_TITLE_LEN):
            errors.append("'title' invalide (1-200 caractères)")

    if "author" in data:
        if not isinstance(data["author"], str):
            errors.append("'author' doit être une chaîne")
        elif not (1 <= len(data["author"].strip()) <= MAX_AUTHOR_LEN):
            errors.append("'author' invalide (1-100 caractères)")

    tags = data.get("tags", [])
    if not isinstance(tags, list):
        errors.append("'tags' doit être une liste")
    elif len(tags) > MAX_TAGS or not all(
        isinstance(t, str) and 1 <= len(t) <= MAX_TAG_LEN for t in tags
    ):
        errors.append("'tags' invalide (max 20, chaque tag 1-50 chars)")

    classification = data.get("classification", "internal")
    if classification not in ALLOWED_CLASSIFICATIONS:
        errors.append("'classification' non autorisée")

    if errors:
        logger.warning("Validation échouée : %s", errors)
        raise ValueError("Payload invalide")

```

```

return Document(
    id=data["id"], title=data["title"].strip(),
    author=data["author"].strip(), tags=tags,
    classification=classification,
)

```

2.3 Cas de test

#	Cas	Payload	Résultat attendu
1	Valide minimal	<code>{"id": 1, "title": "Rapport Q1", "author": "Alice"}</code>	Document créé, classification = internal, tags = []
2	Valide complet	<code>{"id": 2, "title": "Audit", "author": "Bob", "tags": ["finance"], "classification": "confidential"}</code>	Document créé avec tags et classification fournis
3	Champ manquant	<code>{"id": 3, "title": "X"}</code>	<code>ValueError("Payload invalide")</code> — author manquant
4	Type erroné	<code>{"id": "abc", "title": 123, "author": "Bob"}</code>	<code>ValueError("Payload invalide")</code> — id et title de mauvais type
5	Hors allowlist	<code>{"id": 4, "title": "X", "author": "A", "classification": "top_secret"}</code>	<code>ValueError("Payload invalide")</code> — classification non autorisée

Tous les cas invalides (3, 4, 5) lèvent la même exception générique `ValueError("Payload invalide")` côté client, conformément au pattern message générique côté client / détail technique côté logs : le détail (quel champ, quelle règle) n'est journalisé que via `logger.warning`. Le champ `_internal_score` (par exemple un score de confidentialité interne) est exclu de `serialize_document` et n'apparaît jamais dans le JSON produit.

3. TP7.2 — Versioning JSON (compatibilité v1/v2)

Le payload Document évolue d'une version 1 (id, title, author) vers une version 2 qui ajoute tags (liste) et classification (chaîne, allowlist).

3.1 Désérialiseur compatible v1/v2

```

def deserialize_document_v2(raw: str) -> Document:
    """Désérialise un payload v1 ou v2 vers un Document v2."""
    data = json.loads(raw)
    if not isinstance(data, dict):
        raise ValueError("Payload invalide")

    for f in ("id", "title", "author"):
        if f not in data:
            raise ValueError("Payload invalide")

    # Champs v2 absents → valeurs par défaut (compatibilité v1)
    tags = data.get("tags", [])
    classification = data.get("classification", "internal")

    if classification not in ALLOWED_CLASSIFICATIONS:
        raise ValueError("Payload invalide")
    if not isinstance(tags, list) or len(tags) > MAX_TAGS:
        raise ValueError("Payload invalide")

```

```
# Champs inconnus : ignorés, jamais stockés (voir discussion ci-dessous)
KNOWN_FIELDS = {"id", "title", "author", "tags", "classification", "created_at"}
unknown = set(data.keys()) - KNOWN_FIELDS
if unknown:
    logger.info("Champs inconnus ignorés : %s", unknown)

return Document(
    id=data["id"], title=data["title"].strip(),
    author=data["author"].strip(), tags=tags,
    classification=classification,
)
```

Les champs manquants de v1 (tags, classification) reçoivent leurs valeurs par défaut (tags → [], classification → "internal"). Concernant les champs inconnus, le choix retenu est de les tolérer sans planter (ne pas casser un futur v3) mais de ne jamais les stocker aveuglément : ils sont seulement journalisés en niveau info pour suivi, ce qui respecte le principe fail closed sur les champs connus tout en restant tolérant sur les champs non reconnus.

3.2 Matrice de compatibilité

Payload	Lecteur	Exemple	Accepté ?	Raison	Risque
v1	v2	{"id":1,"title":"X","author":"A"}	Oui	Champs manquants → valeurs par défaut (tags=[], classification=internal)	Aucun
v2	v1	{"id":1,"title":"X","author":"A","tags":[],"classification":"public"}	Partiel	Un lecteur v1 ignore tags et classification, qu'il ne connaît pas	Perte d'information si le lecteur v1 ré-émet ensuite ce document (les champs v2 seraient perdus)
v2	v2	{"id":1,"title":"X","author":"A","classification":"top_secret"}	Non	"top_secret" n'est pas dans l'allowlist ALLOWED_CLASSIFICATIONS	Tentative d'injection d'une classification non autorisée par le client
v2 + champ inconnu	v2	{...,"priority":"urgent"}	Oui (ignoré)	Champ priority non reconnu par le contrat v2 actuel : journalisé en info, non stocké, ne fait pas échouer la validation	Faible : le champ est perdu silencieusement ; si ce champ devient important, il faudra l'ajouter formellement au contrat v3 plutôt que de compter sur sa présence implicite

La ligne 'v2 vers v1' illustre un risque réel de perte d'information en cas de relecture/réécriture par un service encore en v1 : c'est un argument supplémentaire pour migrer tous les services consommateurs vers v2 rapidement plutôt que de laisser cohabiter durablement les deux versions. La ligne 'classification: top_secret' montre que la validation par allowlist agit comme un filet de sécurité même pour un payload par ailleurs syntaxiquement et structurellement valide.

4. TP7.3 — Protocol Buffers en Python

4.1 Fichier document.proto

```
// document.proto
syntax = "proto3";
package sgdd;

message Document {
    int32 id          = 1;
    string title       = 2;
    string author      = 3;
    repeated string tags = 4;
    string classification = 5;
    // priority ajouté en v2 – compatible avec les lecteurs v1
    int32 priority     = 6;
}
```

Le fichier est compilé avec `protoc --python_out=. document.proto`, ce qui génère le module `document_pb2.py` utilisé ensuite en Python.

4.2 Script d'encodage/décodage et test de compatibilité

```
import document_pb2
import json

# — Encode (sérialisation) —
doc = document_pb2.Document()
doc.id = 42
doc.title = "Rapport Q1"
doc.author = "Alice"
doc.tags.append("finance")
doc.tags.append("interne")
doc.classification = "confidential"
doc.priority = 2 # champ v2

binary_data = doc.SerializeToString()
print(f"Taille Protobuf : {len(binary_data)} octets")

# — Decode (désérialisation) —
doc2 = document_pb2.Document()
doc2.ParseFromString(binary_data)
print(f"Titre : {doc2.title}, priority : {doc2.priority}")

# — Comparaison de taille avec JSON équivalent —
json_data = json.dumps({
    "id": 42, "title": "Rapport Q1", "author": "Alice",
    "tags": ["finance", "interne"],
    "classification": "confidential", "priority": 2,
}).encode("utf-8")
print(f"Taille JSON : {len(json_data)} octets")
print(f"Ratio : Protobuf ~{len(json_data) / len(binary_data):.1f}x plus petit")
```

Pour le test de compatibilité arrière, un message v2 (contenant `priority = 2`) est sérialisé, puis décodé par un lecteur qui n'accède qu'aux champs `id`, `title`, `author` et `tags` (vision v1) : le décodage réussit sans erreur, le champ `priority` est simplement présent mais non lu par ce lecteur. Cela illustre la rétrocompatibilité native de Protobuf par numéros de champs, sans nécessiter de code de migration particulier.

4.3 Tableau comparatif JSON vs Protobuf

Critère	JSON	Protobuf	Commentaire
Taille payload (octets)	≈ 130 octets (exemple ci-dessus)	≈ 45-55 octets pour le même contenu	Protobuf environ 2 à 3 fois plus compact sur ce petit exemple ; l'écart croît avec des documents plus volumineux et répétitifs
Lisibilité	Excellente (texte, lisible directement)	Illisible sans le .proto (binaire)	JSON facilite le debug manuel ; Protobuf nécessite le schéma pour interpréter les bytes
Validation de types	Aucune intrinsèque (code applicatif requis)	Intrinsèque (types définis dans le .proto)	Protobuf rejette/échoue au niveau binaire si la structure ne correspond pas au schéma
Compatibilité ajout de champ	Possible mais par convention uniquement (TP7.2)	Native, garantie par les numéros de champs	Protobuf gère nativement ce que JSON gère par discipline manuelle de l'équipe
Tooling nécessaire	Aucun (bibliothèque standard json)	protoc + bibliothèque protobuf, fichier .proto partagé	Protobuf impose un outillage et une étape de build supplémentaire

Pour le SGDD, JSON reste le choix par défaut pour l'API publique (interopérabilité, lisibilité, debug facile pour les développeurs front-end), tandis que Protobuf serait pertinent pour les communications inter-services à fort volume (par exemple Document Service → Search Service pour l'indexation), où la compacité et la validation native du schéma apportent un bénéfice mesurable.

5. TP7.4 — Politique de sérialisation sûre (pickle)

5.1 Tableau de politique appliqué au SGDD

Source des données	Format autorisé	Contrôles requis	Justification
API REST (entrée client)	JSON uniquement	Validation stricte (TP7.1), limite de taille, authentification avant traitement	Entrée non fiable par définition → format inerte obligatoire, jamais pickle
Inter-services (Auth ↔ Documents ↔ Search)	JSON ou Protobuf	mTLS, validation à chaque service, signature HMAC pour les messages critiques	Réseau interne ≠ confiance totale (Zero Trust, vu en séance 6)
Cache local (même processus, ex. cache de résultats de recherche)	pickle autorisé	Données générées et consommées localement uniquement, jamais exposées au réseau	Confiance totale : aucune entrée externe n'atteint ce pickle
Fichier uploadé par un utilisateur (futur import de documents)	JSON/CSV/XML via defusedxml	Limite de taille avant parsing, validation stricte, traitement en bac à sable	Jamais pickle sur un fichier fourni par un tiers : risque d'exécution de code
File de messages (événement document_uploaded vers Search, vu en	JSON ou Protobuf	Schéma partagé (registry), validation côté consommateur, signature	Producteurs potentiellement multiples → format interopérable et sûr requis

séance 5)			
Stockage persistant (MongoDB, métadonnées PostgreSQL)	JSON ou Protobuf	Intégrité (HMAC ou chiffrement pour les champs sensibles), plan de migration de schéma documenté	Les données stockées peuvent être altérées entre l'écriture et la lecture (ex. accès direct à la base)

5.2 Checklist 'Safe Serialization Policy' pour le SGDD

- Interdire pickle, marshal et shelve pour toute donnée provenant du réseau (API publique, inter-services, files de messages).
- N'utiliser que `yaml.safe_load()` si un fichier de configuration YAML est lu (jamais `yaml.load()` brut).
- Limiter la taille des payloads JSON/XML avant parsing (par exemple 1 Mo pour les métadonnées, taille spécifique pour les uploads de documents).
- Valider chaque champ après désérialisation : présence, type, longueur, allowlist — comme implémenté en TP7.1 et TP7.2.
- Signer avec HMAC les messages échangés entre Document Service et Search Service via la file de messages, pour détecter toute altération.
- Logger systématiquement les échecs de désérialisation (source, type d'erreur, timestamp) sans jamais inclure le payload brut s'il peut contenir des données sensibles.
- Versionner tous les contrats de données (JSON ou .proto) : ajout de champs optionnels uniquement, jamais de changement de type ou de numéro de champ existant.
- Tester systématiquement les cas limites lors du développement : payload vide, payload géant, types incohérents, champs inconnus, classification hors allowlist.
- Faire apparaître tout usage de `pickle.loads()` comme point de vigilance obligatoire en revue de code, avec justification écrite si exceptionnellement nécessaire.
- Documenter dans chaque service quel(s) format(s) sont acceptés en entrée (JSON v2 uniquement, par exemple) et rejeter explicitement tout autre format avec un code d'erreur clair.

6. Lab sécurité — Threat modeling du flux de sérialisation

6.1 Flux analysé

Le flux étudié est celui d'une requête de création de document dans le SGDD : Client (HTTP) → API Gateway → Sérialisation (encode côté service) → Réseau (TCP) → Désérialisation (decode + validation côté service récepteur).

6.2 Menaces identifiées et contrôles

Menace	Point du flux	Contrôle proposé (appliqué au SGDD)
Tampering (modification du payload en transit)	Réseau	TLS sur l'API publique ; pour les communications inter-services critiques (ex. événement <code>document_uploaded</code>), ajout d'une signature HMAC vérifiée par Search Service avant traitement
Injection de champs (ex. classification: "top_secret" ou un futur champ <code>is_admin</code>)	Client → API	Validation stricte avec allowlist de champs et de valeurs (TP7.1) : tout champ ou valeur non prévu fait échouer la désérialisation (sauf champs explicitement tolérés et ignorés comme <code>priority</code> en v2)

Replay (renvoi d'un payload de création de document légitime capturé)	Réseau	Idempotency-Key (déjà définie en séance 6 pour POST /documents) combinée à un contrôle de fraîcheur (timestamp) pour les échanges inter-services signés par HMAC
Fuite dans les logs (champ _internal_score ou tokens dans les logs de désérialisation)	Sérialisation / désérialisation	Exclusion explicite des champs sensibles dans <code>serialize_document</code> (TP7.1) ; les logs de validation n'incluent que la liste des erreurs, jamais le payload complet
Payload géant (DoS par épuisement mémoire)	Client → API	Vérification de Content-Length / taille du payload avant tout appel à <code>json.loads()</code> , conformément à la limite définie dans le contrat d'API (séance 6)
Exécution de code (pickle/YAML non sûr sur une entrée non fiable)	Désérialisation	Interdiction stricte de <code>pickle.loads()</code> sur toute entrée réseau (TP7.4) ; seuls des formats inertes (JSON, Protobuf) sont utilisés pour les entrées non fiables
Type confusion (ex. id envoyé comme chaîne au lieu d'entier)	Désérialisation	Vérification de type stricte avec <code>isinstance()</code> après le parsing JSON (TP7.1), sans coercion implicite de type

6.3 Checklist 'Secure Deserialization Baseline' — état pour le SGDD

- Formats inertes uniquement (JSON, Protobuf) pour les entrées non fiables — conforme (TP7.1, TP7.3).
- `pickle`, `yaml.load()`, `eval()`, `exec()` interdits sur entrées réseau — conforme, formalisé dans la politique TP7.4.
- Limite de taille avant parsing — à appliquer systématiquement au niveau du Gateway et de chaque service, en cohérence avec les limites du contrat d'API.
- Validation de la structure racine (dict) et des champs attendus (allowlist) — conforme (TP7.1, TP7.2).
- Validation stricte des types par champ (`isinstance`) — conforme (TP7.1).
- Validation des valeurs (bornes, longueurs, allowlist pour les enums comme classification et role) — conforme (TP7.1).
- Signature HMAC ou TLS mutuel pour les échanges inter-services sensibles — à généraliser à la file de messages `document_uploaded`.
- Vérification de l'authentification/autorisation avant désérialisation complète — cohérent avec le contrat d'API de la séance 6 (AuthN/AuthZ avant traitement).
- Logs des échecs de désérialisation sans données sensibles — conforme (logger.warning sur la liste d'erreurs, jamais le payload).
- Timeout sur le parsing — à ajouter explicitement pour le parsing XML si un import de documents XML est introduit ultérieurement.
- Rejet par défaut (fail closed) de tout payload non conforme — conforme, c'est le comportement de `deserialize_document` et `deserialize_document_v2`.
- Tests réguliers avec des payloads malformés — couvert partiellement par les 5 cas de test du TP7.1, à étendre avec des payloads volumineux et des champs inconnus multiples.

7. Synthèse

Ce TP a permis d'implémenter concrètement, pour le service Document du SGDD, un contrat JSON strict avec validation exhaustive et approche fail closed (TP7.1), un mécanisme de versioning v1/v2 par ajout de champs optionnels avec une matrice de compatibilité justifiée (TP7.2), une comparaison pratique entre JSON et Protobuf montrant les gains de compacité et de validation native de ce dernier (TP7.3), et une politique de sérialisation sûre excluant pickle de tout flux non fiable, avec une checklist applicable à l'ensemble du projet (TP7.4).

Le point central de la séance, appliqué au SGDD, est que la validation syntaxique et typée d'un JSON (ce qu'effectue `deserialize_document`) est nécessaire mais non suffisante : elle doit être complétée par l'authentification de l'origine, l'autorisation de l'action demandée, et le contrôle des valeurs métier via des allowlists. Le threat modeling du flux de sérialisation (section 6) a confirmé que chaque transformation de données (encode ou decode) est un point où un attaquant contrôlant les bytes peut tenter d'influencer le comportement du système — d'où l'importance de n'utiliser que des formats inertes (JSON, Protobuf) pour toute donnée non fiable, et de réserver pickle aux échanges en confiance totale et strictement locaux.